

Resolución y Guía de Corrección: TP Algoritmos Greedy

Asignatura: Programación III | Unidad: Estrategias de Diseño de Algoritmos (Voraces)

Parte 1: Formalización Teórica y Pseudocódigo

1. Definición Formal de los 5 Componentes del Algoritmo Voraz

El alumno debe identificar explícitamente los 5 elementos del modelo general Greedy:

- **Conjunto de Candidatos (C):**

Conjunto finito de monedas o billetes de las denominaciones disponibles $D = \{d_1, d_2, \dots, d_k\}$, con disponibilidad ilimitada de cada tipo.

- **Función de Selección:**

Selecciona la denominación $d_i \in C$ de mayor valor absoluto tal que no supere el monto restante por devolver (criterio local voraz).

- **Función de Factibilidad:**

Dado un acumulado de cambio S y una moneda d_i , verifica si $(S + d_i) \leq v$ (donde v es el cambio total solicitado).

- **Función Objetivo:**

Minimizar la cantidad total de monedas/billetes entregados: **Minimizar** $\sum x_i$, donde x_i es la cantidad de monedas de la denominación d_i .

- **Función Solución:**

Verifica si la suma acumulada de las monedas seleccionadas es exactamente igual al monto requerido: $\sum (x_i \cdot d_i) = v$.

2. Implementación Recomendada en Pseudocódigo

Solución optimizada utilizando división entera y módulo para evitar iteraciones unitarias:

```
Funcion cambioGreedy(monto: Entero, D: Arreglo de Enteros) -> (Entero, Diccionario) //
Precondición: D está ordenado de forma estrictamente decreciente solucion = DiccionarioVacio()
totalMonedas = 0 montoRestante = monto Para i = 0 Hasta longitud(D) - 1 Hacer Si D[i] <=
montoRestante Entonces cantidad = montoRestante DIV D[i] solucion.agregar(D[i], cantidad)
totalMonedas = totalMonedas + cantidad montoRestante = montoRestante MOD D[i] FinSi Si
montoRestante == 0 Entonces Romper // Solución alcanzada FinSi FinPara Retornar (totalMonedas,
solucion) FinFuncion
```

Criterio de Corrección: Si el alumno implementa el algoritmo restando de a 1 moneda repetidamente mediante un `Mientras`, es conceptualmente válido pero menos eficiente. Valore positivamente el uso de operaciones numéricas (`DIV` / `MOD`).

Parte 2: Análisis de Casos de Prueba y Optimalidad

1. Simulación del Sistema B con Cambio \$6 ($D_B = [4, 3, 1]$)

- **Resultado obtenido por el Algoritmo Greedy:**

1. Selecciona la moneda más grande factible: $d = 4$. Queda restando $6 - 4 = 2$.

2. Para \$2, la moneda de 4 o 3 no es factible. Toma $d = 1$. Queda restando $2 - 1 = 1$.

3. Toma nuevamente $d = 1$. Queda restando \$0.

Resultado Voraz: 3 monedas entregadas (1 de \$4, 2 de \$1).

• **Solución Óptima Global:**

Entrega 2 monedas de \$3 ($3 + 3 = 6$).

Resultado Óptimo: 2 monedas.

Demostración de Suboptimalidad: El ejemplo prueba de forma directa que el algoritmo Greedy **NO siempre garantiza la solución óptima global** para cualquier sistema monetario.

2. Justificación Teórica: ¿Por qué falla en el Sistema B pero funciona en el Sistema A?

• **Miopía del Enfoque Voraz:** Los algoritmos Greedy toman decisiones locales óptimas en cada paso sin reevaluar ni hacer *backtracking* (reconsiderar decisiones pasadas). Al tomar la moneda de \$4 en el primer paso, la estrategia Greedy queda "atrapada" e imposibilitada de formar la combinación de dos monedas de \$3.

• **Propiedad de Canonicidad del Sistema Monetario:**

El **Sistema A** ($[100, 50, 20, 10, 5, 2, 1]$) es un *Sistema Monetario Canónico*. En estos sistemas, cada denominación es un múltiplo o cumple relaciones de divisor/combinación respecto a las anteriores que garantizan la propiedad de **subestructura óptima**.

El **Sistema B** ($[4, 3, 1]$) es un *Sistema No Canónico*. Para estos sistemas, el problema del cambio requiere **Programación Dinámica** (o Búsqueda en Amplitud/BFS) para asegurar el óptimo en complejidad $O(k \cdot v)$.

3. Análisis de Complejidad Temporal

• **Si el vector de denominaciones ya viene ordenado:**

La complejidad es $O(k)$, donde k es el número de denominaciones distintas en el sistema monetario. Esto se debe a que recorre el arreglo de denominaciones una sola vez (si usa `DIV`/MOD``).

• **Si el vector requiere ordenamiento previo:**

Sumará el costo de ordenar las k denominaciones de mayor a menor: $O(k \cdot \log k)$.

• **Complejidad Independiente del Monto:**

A diferencia de la versión por restas sucesivas o la solución por Programación Dinámica, la versión optimizada con aritmética no depende del valor numérico del monto v .

Tabla de Puntuación Sugerida

Sección	Ítem a Evaluar	Puntaje Sugerido
Parte 1.1	Definición precisa de los 5 componentes formalizados.	20%
Parte 1.2	Pseudocódigo sintácticamente correcto y lógico.	20%
Parte 2.1	Ejecución manual correcta mostrando el caso de suboptimalidad (3 vs 2 monedas).	20%
Parte 2.2	Explicación teórica de la miopía voraz y sistemas canónicos.	25%
Parte 2.3	Cálculo y justificación correcta de la complejidad temporal $O(k)$ o $O(k \log k)$.	15%